

HIGH LEVEL DESIGN (HLD)

Project Name: Inventra - Inventory Management System

Prepared By: Team Codeverse

Date: 1 March 2026

Project Duration: 1 month (Design, Development, Testing & Deployment)

Version : 1.0

Three Tier Architecture Diagram



- **Architecture pattern**
Inventra uses a **monolithic architecture** pattern, where all components UI, business logic, and data access are integrated into a single unified application for simplicity and easier management in this project scope.

Tech Stack

Selected Technologies for Retail Management System

Frontend Stack

- **React 18:** Component-based UI library for dynamic dashboards and billing interfaces
- **TypeScript:** Type safety and better maintainability for complex forms and calculations
- **React Router:** Client-side routing for multi-page navigation (Dashboard, Billing, Inventory, Customers)
- **Axios:** REST API communication with Spring Boot backend
- **Tailwind CSS / Material UI:** Responsive and modern UI design
- **React PDF (@react-pdf/renderer):** GST-compliant invoice and report generation
- **Vite:** Fast development server and optimized production builds

Backend Stack

- **Spring Boot 3:** RESTful backend framework

- **Spring Data JPA (Hibernate):** ORM for database interaction
- **Spring Security:** Role-based authentication & authorization
- **JWT (JSON Web Token):** Stateless user authentication
- **BCrypt:** Secure password hashing
- **Bean Validation (Jakarta Validation):** Server-side input validation
- **Lombok:** Cleaner entity and DTO code

Backend Responsibilities:

- GST calculation logic
- Invoice processing
- Inventory updates
- Customer & supplier management
- Business rule enforcement
- Secure API endpoints

Database

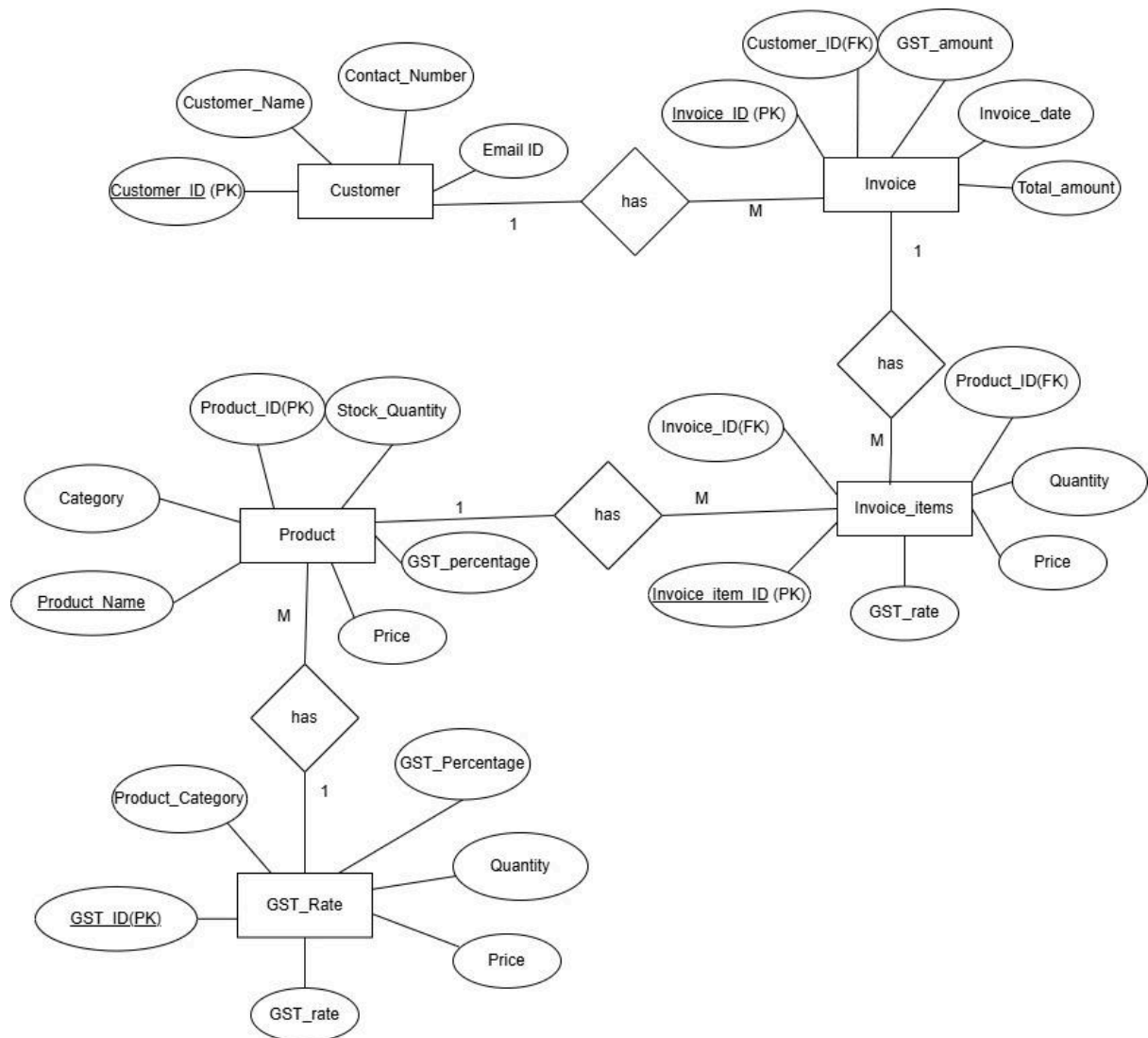
- PostgreSQL 15: Relational database for structured retail data
- pgAdmin: Database management

Security Architecture

- JWT-based authentication
- Role-based access (Admin / Accountant / Staff)
- CORS configuration
- BCrypt password encryption

- Backend-side GST validation

Entity Relationship ER



RestFul API Endpoints

Authentication & Password

POST /api/v1/auth/register

POST /api/v1/auth/login

POST /api/v1/auth/refresh

POST /api/v1/auth/logout

POST /api/v1/auth/password/forgot

GET /api/v1/auth/password/validate?token=

POST /api/v1/auth/password/reset

User Management

GET /api/v1/users

GET /api/v1/users/{id}

PUT /api/v1/users/{id}

DELETE /api/v1/users/{id}

Customers

POST /api/v1/customers

GET /api/v1/customers

GET /api/v1/customers/{id}

PUT /api/v1/customers/{id}

DELETE /api/v1/customers/{id}

Products & Inventory

POST /api/v1/products

GET /api/v1/products

GET /api/v1/products/{id}

PUT /api/v1/products/{id}

DELETE /api/v1/products/{id}

GET /api/v1/inventory

PUT /api/v1/inventory/{productId}

Payments (Razorpay)

POST /api/v1/payments/create-order

POST /api/v1/payments/verify

GET /api/v1/payments/invoice/{invoiceId}

Analytics & Reports

GET /api/v1/analytics/summary

GET /api/v1/analytics/revenue?period=daily

GET /api/v1/analytics/revenue?period=weekly

GET /api/v1/analytics/revenue?period=monthly

GET /api/v1/analytics/sales?startDate=&endDate=

HLD: Security Architecture

1 Identity & Access Management

- JWT Stateless Auth
- Short-Lived Access Tokens
- Refresh Token Rotation
- Role-Based Access (RBAC)
- BCrypt Password Hashing

- Brute-Force Protection
- IP Rate Limiting
- Secure Cookie Storage

2 Data Protection Strategy

- TLS 1.3 Encryption
- AES-256 At Rest
- Encrypted DB Backups
- Environment Secret Config
- No Hardcoded Credentials
- Sensitive Data Masking
- Unique DB Constraints
- Foreign Key Enforcement

3 Application Security Controls

- Parameterized Queries Only
- Hibernate ORM Usage
- DTO Validation Layer
- Global Exception Handler
- XSS Input Sanitization
- CSP Security Headers
- CSRF Token Protection
- Controlled Error Responses

4 Monitoring & Audit

- Structured Log Framework
- Log Level Segregation
- Sensitive Log Redaction
- User Activity Tracking
- Anomaly Detection Flags
- Admin Audit Access
- API Rate Limiting

LLD – Security & Authentication Module

1. Authentication Flow – Detailed Sequence

1.1 Registration Flow (User Creation)

Endpoint: `POST /auth/register`

Flow:

1. Controller receives `RegisterRequestDTO`

2. DTO validated using `@Valid`
3. Service Layer Checks:
 - Email uniqueness
 - Role existence
4. Password hashed using:
`BCryptPasswordEncoder(strength = 12)`
5. User entity persisted via JPA Repository
6. Audit log entry created:
 - `action_type = REGISTER`
7. Response returned with HTTP 201 CREATED

1.2 Login Flow

Endpoint: `POST /auth/login`

Flow:

1. AuthenticationManager validates credentials
2. If authentication succeeds:
 - Generate Access Token (validity: 15–30 minutes)
 - Generate Refresh Token (validity: 7 days)

Token Properties:

- Signed with HMAC-SHA256
- Claims include:

- userId
- role
- email

Tokens stored in:

- httpOnly
- Secure
- SameSite=Strict cookies

3. Login activity stored in audit_log

If authentication fails:

- Increment failed_attempt counter
- Lock account after configured threshold

1.3 Refresh Token Flow

1. Extract refresh token from cookie
2. Validate signature
3. Check expiration
4. Issue new access token
5. Rotate refresh token (recommended best practice)

2. Authorization Layer Design

2.1 Spring Security Configuration Flow

SecurityFilterChain



JwtAuthenticationFilter



UsernamePasswordAuthenticationFilter

The security layer intercepts all requests before reaching the controller.

2.2 Role Enforcement

At Controller Level:

```
@PreAuthorize("hasRole('ADMIN')")
```

At Service Level:

- Validate business constraints
- Prevent horizontal privilege escalation

3. JWT Utility Design

3.1 JwtService Responsibilities

- generateAccessToken(UserDetails)
- generateRefreshToken(UserDetails)
- extractUsername(token)
- validateToken(token)

3.2 JWT Token Structure

Header

alg: HS256

typ: JWT

Payload

sub: user email

role: ADMIN

iat: timestamp

exp: timestamp

Signature

HMACSHA256(base64(header.payload), secret)

Secret stored in environment variable:

JWT_SECRET

4. Password Reset – Internal Working

4.1 Forgot Password Flow

1. Verify email exists
2. Generate secure random token:
 - UUID + timestamp hash
3. Store:
 - password_reset_token
 - expiry_time (15 minutes)
4. Send reset link via email

4.2 Reset Password Flow

1. Validate token existence
2. Validate expiry
3. Hash new password
4. Invalidate token
5. Create audit entry

5. Database-Level Security (LLD View)

5.1 Constraints

- UNIQUE(email)
- UNIQUE(invoice_number)
- UNIQUE(gst_number)
- Foreign keys with ON DELETE RESTRICT

5.2 Indexing

- email
- invoice_date
- customer_id
- product_id

5.3 Connection Pool

- HikariCP
- Configured max pool size
- Defined idle timeout

6. Global Exception Handling

Centralized using:

@ControllerAdvice

Handles:

- MethodArgumentNotValidException
- AccessDeniedException
- BadCredentialsException
- EntityNotFoundException
- CustomBusinessException

Standard API Error Response:

```
{  
  "timestamp": "",  
  "status": 400,  
  "error": "Validation Error",  
  "message": "Email already exists",  
  "path": "/api/v1/auth/register"  
}
```

No stack trace exposed.

7. Audit Logging – Internal Mechanism

On every critical action:

Before update:

- Capture old state (JSON)

After update:

- Capture new state

Stored in:

audit_log

user_id

entity_name

action_type

old_value (JSONB)

new_value (JSONB)

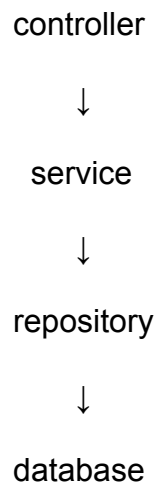
timestamp

ip_address

Logged via:

- AOP-based interceptor
or
- Service-layer logging method

8. Module Dependency Structure



Security layer intercepts before controller.

Final LLD Classification

This system implements:

- Stateless token-based authentication
- Multi-layer role enforcement
- Secure password lifecycle management
- Verified payment processing
- Transaction-safe business operations
- Centralized exception management
- Structured audit logging

This is designed keeping in mind the best industry practices suitable for small-scale SaaS deployment.